

APEX INSTITUTE OF MANAGEMENT · CHANDIGARH UNIVERSITY

Agentic AI

From Intelligent Response to Autonomous Action

Resource Person: **Muzamil Aslam**

2-Hour Session

Today's Agenda

A two-hour session structured to build understanding step by step — from core definitions to the internal mechanics that power real autonomous AI systems.

Hour 1 — Foundations

What is an Agent · What is Agentic AI · How it Works · The Agent Lifecycle

Hour 2 — Deep Dive

Types of Agents · Planning & Execution Mechanisms · Memory & Contextual Intelligence

Throughout

Live diagrams and real-world examples connecting theory to working systems

Closing

Summary of key takeaways followed by open discussion and Q&A

Introduction to Agentic AI

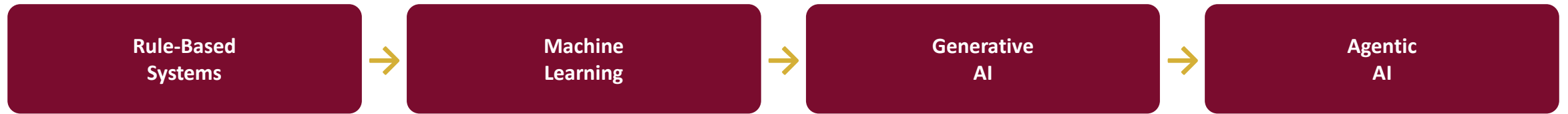


AI has evolved through distinct eras: from rule-based systems that only followed instructions, to Generative AI that creates content on request, to today's frontier — Agentic AI, where systems independently decide what to do, plan how, and act to achieve a goal.

- ▶ Agentic AI marks the shift from AI that responds to AI that acts
- ▶ It combines reasoning (LLMs), memory, tools, and autonomy into one system
- ▶ Powers products like AI coding assistants, research agents, and automated workflows
- ▶ This session builds the theory needed to design and evaluate such systems

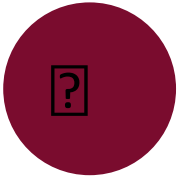
From Automation to Agentic AI

Understanding Agentic AI requires seeing where it fits in the broader evolution of intelligent systems.



- ▶ Rule-Based: fixed if-then logic, no learning
- ▶ Machine Learning: learns patterns from data, still needs human-defined tasks
- ▶ Generative AI: creates text, images, and code from prompts (e.g., ChatGPT)
- ▶ Agentic AI: sets sub-goals, uses tools, remembers context, and acts with minimal supervision

What is an Agent?



In AI and computer science, an agent is any entity that perceives its environment through sensors and acts upon that environment through actuators to achieve a specific goal.

- ▶ Classic definition: “anything that can perceive its environment and act upon it” (Russell & Norvig)
- ▶ Examples beyond AI chatbots: a thermostat, a self-driving car, a trading bot
- ▶ An agent operates in a continuous cycle: Perceive → Decide → Act
- ▶ The 'intelligence' of an agent depends on how well it decides, not just how it acts

Key Properties of an Agent

Four defining properties distinguish a true agent from a simple program — plus two more that separate strong agents from basic ones.

Autonomy

Operates without continuous human control once given a goal

Reactivity

Perceives its environment and responds to changes in real time

Proactiveness

Takes initiative to pursue goals, not just react to events

Social Ability

Interacts with other agents, tools, APIs, or humans as needed

Persistence

Continues operating over time, not a single one-shot execution

Adaptability

Adjusts its plan when the environment or goal changes

Agent vs. Traditional Software

The contrast below highlights why agents represent a fundamentally different design philosophy.

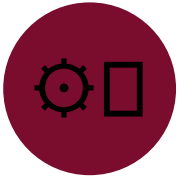
Traditional Software

- Follows fixed instructions
- No independent decision-making
- Fails silently on unexpected input
- Requires explicit code for every case

AI Agent

- Sets and pursues goals
- Reasons about the best next step
- Adapts to unexpected situations
- Learns / updates its approach using context

What is Agentic AI?



Agentic AI refers to AI systems — typically built on Large Language Models (LLMs) — that can independently plan, decide, use tools, and take multi-step actions to accomplish a goal with minimal human intervention.

- ▶ It is not a single model, but an architecture: LLM + Memory + Tools + Planning logic
- ▶ Given a goal like 'book me the cheapest flight,' the agent breaks it down and executes it
- ▶ Distinguishes itself through goal persistence — it keeps working until the objective is met
- ▶ Forms the foundation of AI copilots, research assistants, and autonomous workflow bots

Agentic AI vs Generative AI vs Traditional AI

Traditional AI

- Rule / logic based
- Narrow task
- No learning from new data
- Deterministic outputs

Generative AI

- Produces text / image / code on prompt
- Single-turn response
- No independent action
- Human drives every step

Agentic AI

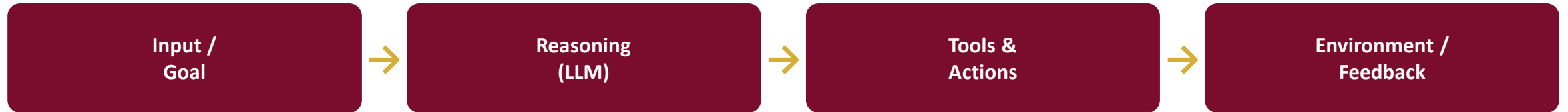
- Plans multi-step tasks
- Uses tools & memory
- Acts autonomously
- Pursues goals over time

Core Characteristics of Agentic AI

- ▶ Goal-Directed: works toward a defined outcome, not just a single reply
- ▶ Autonomous: chooses its own steps without step-by-step instructions
- ▶ Tool-Using: calls external APIs, databases, or software to extend its ability
- ▶ Context-Aware: uses memory to maintain continuity across a task
- ▶ Reflective: can evaluate its own output and self-correct
- ▶ Multi-Step: executes a sequence of actions, not a single-shot response

How Agentic AI Works — Architecture

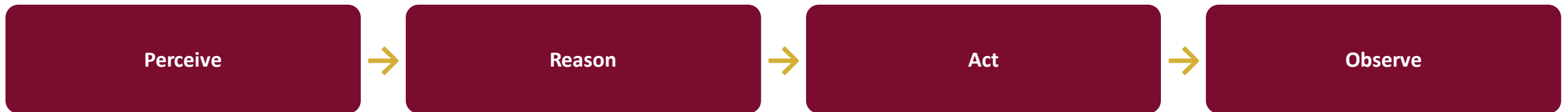
An Agentic AI system is built from four cooperating components that together enable autonomous behaviour.



- Input: the user's goal or an environmental trigger enters the system
- Reasoning Core (LLM): interprets the goal and plans the next step
- Tools / Actions: the agent executes real actions — search, code, API calls
- Feedback: results flow back in, updating the agent's next decision

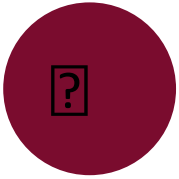
The Perceive–Reason–Act Loop

At the heart of every agent is a repeating control loop, executed continuously until the goal is achieved.



- ▶ Perceive: gather current state — user input, sensor data, tool output
- ▶ Reason: the LLM evaluates the state against the goal and picks the next action
- ▶ Act: the chosen action is executed (a tool call, a message, a code run)
- ▶ Observe: the outcome feeds back into perception, and the loop repeats

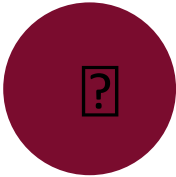
The LLM as the Reasoning Engine



The Large Language Model is the 'brain' of the agent — it does not act directly on the world, but decides what should happen next.

- ▶ Interprets natural-language goals and breaks them into actionable steps
- ▶ Selects which tool or function is appropriate for the current step
- ▶ Uses techniques like Chain-of-Thought to reason before acting
- ▶ Its quality directly determines how well the agent plans and recovers from errors

Tools, APIs & Function Calling



An LLM alone can only generate text — tools are what let an agent actually affect the world.

- ▶ Function calling: the LLM outputs a structured request (e.g., `search_web(query)`)
- ▶ The agent framework executes that function and returns the result to the LLM
- ▶ Common tools: web search, code execution, database queries, calendar / email APIs
- ▶ This tool layer is what turns a chatbot into a system capable of real-world action

AI Agent Lifecycle — Overview

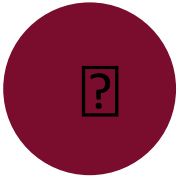
Every agent, regardless of task, moves through four broad lifecycle stages from receiving a goal to completing it.



The lifecycle is cyclical, not linear — agents often revisit earlier stages.

We examine each stage in detail over the next four slides.

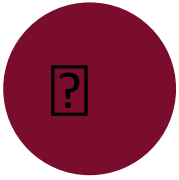
Perception & Input



The lifecycle begins with the agent gathering everything it needs to understand the current situation.

- Receives the user's goal or task instruction
- Pulls in relevant context: conversation history, files, sensor / API data
- Filters noise and identifies what is actually relevant to the goal
- Quality of perception directly limits the quality of every later stage

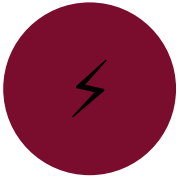
Reasoning & Planning



With the situation understood, the agent must decide on a course of action.

- Breaks the overall goal into smaller, manageable sub-tasks
- Considers available tools and past experience (memory) when choosing steps
- May generate multiple candidate plans and select the most promising one
- Uses reasoning techniques such as Chain-of-Thought or ReAct (covered later)

Action & Execution



The plan is put into motion — this is where the agent interacts with the real world.

- ▶ Executes tool calls, API requests, or code as decided in the planning stage
- ▶ Monitors execution in real time — some steps may run in parallel or sequence
- ▶ Handles errors gracefully: retries, alternative tools, or asks for clarification
- ▶ Produces intermediate results that feed the next reasoning cycle

Feedback & Learning



After acting, the agent evaluates the outcome and updates its understanding before the loop repeats.

- Compares the actual result against the intended goal
- Stores useful outcomes in memory for future reasoning
- Adjusts its plan if the goal is not yet achieved, looping back to reasoning
- Over many cycles, this feedback improves the agent's effectiveness

Types of Agents — Overview

Agents differ in how much reasoning they perform before acting. We will explore four major categories.

Reactive Agents

Act instantly on current input, no internal planning

Deliberative Agents

Build and follow an explicit plan before acting

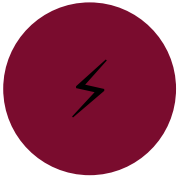
Hybrid Agents

Combine fast reaction with deeper planning as needed

Multi-Agent Systems

Multiple specialised agents collaborating on one goal

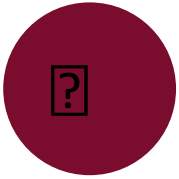
Reactive Agents



The simplest category: these agents map perceptions directly to actions using fixed condition–action rules, with no internal model of the world.

- ▶ Respond immediately based on current input — 'if X, then do Y'
- ▶ No memory of the past and no planning ahead
- ▶ Fast and predictable, but cannot handle novel or complex situations
- ▶ Example: a simple customer-support bot answering FAQs with fixed replies

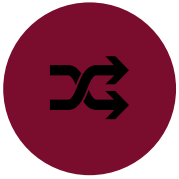
Deliberative (Planning) Agents



These agents build an internal model of the world and reason before acting, enabling them to handle complex, multi-step goals.

- ▶ Maintain an internal representation of the current state and desired goal
- ▶ Generate a sequence of actions (a plan) before executing anything
- ▶ Can evaluate trade-offs between different possible action sequences
- ▶ Example: an AI travel-planning agent sequencing flights, hotels, and transport

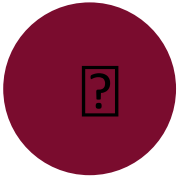
Hybrid Agents



Hybrid agents combine the speed of reactive behaviour with the depth of deliberative planning, switching modes as the situation demands.

- ▶ Use reactive responses for routine, time-sensitive situations
- ▶ Switch to deliberative planning when the task is complex or novel
- ▶ Most modern LLM-based agents (e.g., coding assistants) are hybrid in nature
- ▶ Balances responsiveness with the ability to solve harder problems

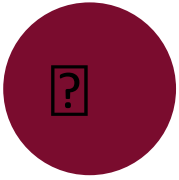
Multi-Agent Systems



Instead of one agent doing everything, complex tasks can be divided among several specialised agents that communicate and collaborate.

- ▶ Each agent has a defined role — e.g., 'Researcher', 'Coder', 'Reviewer'
- ▶ Agents coordinate via message passing or a shared orchestrator
- ▶ Enables division of labour and parallel processing of sub-tasks
- ▶ Example: an AI software team where one agent writes code and another tests it

Planning & Execution Mechanisms



Once an agent decides to plan, it needs concrete mechanisms to turn a goal into an ordered set of executable actions.

- ▶ Planning transforms a vague goal into a structured, step-by-step approach
- ▶ Execution mechanisms manage how those steps are actually carried out
- ▶ Together they determine an agent's reliability on complex, multi-step tasks
- ▶ Next: two widely used mechanisms — Task Decomposition and the ReAct framework

Task Decomposition & Chain-of-Thought

Complex goals are rarely solved in one step — they must be broken down.

Task Decomposition

Splitting a large goal into smaller, ordered sub-goals.

Example: 'Plan a conference' → venue, budget, speakers, schedule, promotion

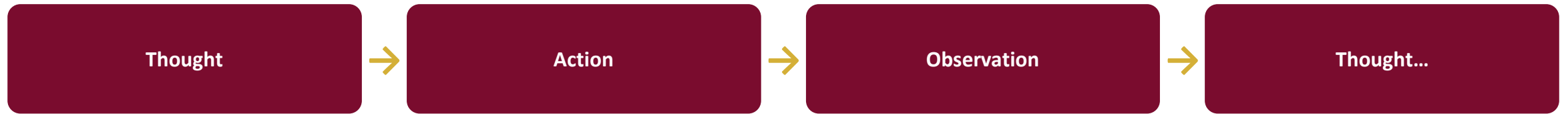
Chain-of-Thought (CoT)

The model reasons step-by-step in text before answering.

Improves accuracy on multi-step problems by making intermediate reasoning explicit

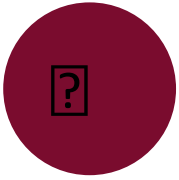
The ReAct Framework

ReAct (Reason + Act) is a widely used pattern that interleaves reasoning with action, rather than planning everything upfront



- ▶ The agent generates a 'Thought' explaining its next intended step
- ▶ It then takes an 'Action' (a tool call) based on that thought
- ▶ The result becomes an 'Observation,' which informs the next Thought
- ▶ This loop continues until the goal is reached — enabling adaptive, error-correcting behaviour

Memory & Contextual Intelligence



Memory allows an agent to maintain continuity, learn from experience, and behave intelligently across a long or multi-session task rather than treating every step as brand new.

- ▶ Without memory, an agent forgets everything after each response
- ▶ Memory allows personalization, consistency, and learning from past actions
- ▶ Contextual intelligence = combining memory with real-time perception to act appropriately
- ▶ Critical for agents handling long, multi-step, or ongoing tasks

Short-Term vs Long-Term Memory

Agent memory is typically designed in layers, similar to human cognition.

Short-Term Memory

- Holds the current conversation / task context
- Limited by the LLM's context window
- Cleared or summarized after the task ends

Long-Term Memory

- Persists across sessions
- Often stored in a vector database
- Retrieved via similarity search (RAG) when relevant

Summary, Examples & Key Takeaways

Agentic AI represents the shift from AI that answers to AI that acts — combining reasoning, tools, memory, and autonomy into systems that pursue goals independently.

- ▶ Real-world examples: AI coding agents, autonomous research assistants, workflow-automation bots, end-to-end customer-service agents
- ▶ Remember: Agent = Perceive + Reason + Act; Agentic AI adds planning, tools, and memory
- ▶ Lifecycle: Perception → Planning → Execution → Learning (repeats)
- ▶ Design choice: Reactive vs Deliberative vs Hybrid vs Multi-Agent depends on task complexity

Thank You · Muzamil Aslam · Apex Institute of Management, Chandigarh University · Open for Q&A